Optimizing sEMG-Based Typing Prediction: A Comparative Analysis of Recurrent, Transformer and Residual Neural Architectures for Personalized, Lightweight Models

Yamm Elnekave
Department of Computer Science
University of California-Los Angeles
Los Angeles, CA 90095
yamme@ucla.edu

Amit Rand
Department of Mathematics
University of California-Los Angeles
Los Angeles, CA 90095
amit.rand@ucla.edu

Ido Dukler
Department of Electrical and Computer Engineering
University of California-Los Angeles
Los Angeles, CA 90095
idukler@g.ucla.edu

Abstract

Meta and Reality Labs' *emg2sqwerty* project use electrode wristbands to collect sEMG data for the task of predicting typing on a keyboard only from motor neuron activations. We propose a survey of alternative models that meet and exceed the current out-of-the-box implementations made available by Meta. Given the time dependence relation of typing data, our exploration consisted of recurrent and residual architectures. We found ResNet based architectures for feature extracting and encoding trained on sEMG data from a single user to perform best for the task of predicting typed characters.

1 Introduction

Surface electromyography (sEMG) has been shown to be an effective and non-invasive method for capturing neural activity by measuring electrical signals generated from muscle activations. Meta recently introduced emg2qwerty, a dataset demonstrating that these signals hold strong potential for extracting semantic information solely from motor neuron activations [1]. These signals contain semantic information capable of accurately predicting typing inputs solely from these motor neurons. This capability opens a multitude of applications with brain-computer interfaces (BCI) and human-computer interaction (HCI) in general, first and foremost through assistive systems and technologies for people with motor disabilities [2] [3].

Research has shown that sEMG based technique are particularly useful for healthcare applications, specifically in providing seamless interactions between humans and digital systems. A key example is neuro-rehabilitation, where sEMG-drive BCIs has been shown to significantly improve a patients outcome through personalized therapeutic interventions that allow for patients to regain their control and independence [2]. Even more so, the predictive capabilities provided through sEMG signals extends to other important tasks such as hand pose estimation and accurate gesture recognition.

Motivated by the need for fast, accurate, and personalized models, our study investigates Neural Network architectures with the specific purpose of single-user scenarios. These type of patient

specific models are very important as previously mentioned in neuro-rehabilitation, as each patient's sEMG signals can vary considerably, requiring rapid and individualized training. Additionally, we aim to develop a compact and efficient architecture suited for deployment on everyday devices, to improve accessibility and usability in real-world environments. Studies have shown that models within the range of 1M-10M parameters in size fit the criteria for usability [4]. In contrast to large-scale computationally intensive models with hundreds of millions of parameters, our goal is to deliver accurate, lightweight models that can be trained efficiently and deployed easily, to help improve the practical use of personalized sEMG based systems.

2 Methods

2.1 Architectures

Our experimentation explored different neural architectures, each motivated by different intuitions about the nature of sEMG signals and both their spatial and temporal dynamics. At a high level, we hypothesize that a single input sample into our model (which is about one second of sEMG data) contains relevant information about muscular activations that contain the information of movements preceding a key press, the key press itself, and transitions between sequential characters.

We isolated specifically the Encoder and Feature Extractor parts of the architecture to best tackle the problem in a controlled scenario and understand the impact of each modification.

Recurrent Architectures (LSTM and GRUs): Typing characters on a keyboard over time to generate words and sentences indicate both semantic and temporal dependencies. Semantic from how some characters strung together generate meaningful words and temporal from the physical action of typing these characters on a keyboard over time. We hypothesized that recurrent architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs) would effectively capture both the semantic and temporal dependencies present in typing. In addition, we intuit that motor activations occurring immediately before and after a character is typed provided critical context necessary for getting an accurate prediction of the pressed key. The inherent memory state of LSTMs and GRUs capable of selectively remembering and forgetting historical information based on relevance is best suited for learning these types of temporal relationships. This model then learns the different motor activation representations that lead up to and follow a particular key press.

Transformer Architectures: Transformers have recently achieved state-of-the-art results across a large range of both sequential and attention based tasks. We wanted to explore transformer encoders due to their potential to selectively pay attention to relevant temporal segments of the sEMG data which represent the most informative parts of the sEMG motor activation of a key press and their relationship to other segments of the 1 seconds signal. Through their self-attention mechanisms, they could dynamically weight specific parts of the input signal according to relevance in predicting each character.

ResNet Feature Extractors (ResFEX): Recognizing that sEMG data has very complex local activation patterns that are crucial for accurate character prediction, we introduced a Residual Network (ResNet) architecture for our feature extractor. Our intuition was that ResNet's ability to learn residual representations that related both subtle changes within the data and local features could capture spatial-temporal patterns in the raw sEMG signals very well. The idea behind residual connections was their proven capability to train deeper networks by preventing vanishing gradient issues and propagating the signal within each layer of the network, thereby allowing the network to learn crucial mappings or transformations as residual functions relative to the original input.

Architecture notes for reconstruction: Our ResNet architectures had an additional downsampling layer as that allowed us to have the ResNet input and output layer to be the same size. Additionally, the LSTM modules only take three arguments: batch size, sequence length, and input size. Thus some reshaping of the inputs was necessary to create a valid feature extractor.

2.2 Training

Due to computational constraints and time limitations, we decided to standardize a constrained training protocol across all experiments. All models were limited the a similar order of magnitude in trainable parameter size to conduct a fair comparison. Each model was trained for a fixed

duration of 25 epochs, only after it was selected based on preliminary experimentation coupled with observing sufficient initial convergence and considerable relative difference in performance. This approach allowed for an efficient comparison between architectures while managing computation resources effectively. Further optimization of model performance with thorough hyperparameter tuning, learning rate scheduling, etc. was reserved for the best-preforming overall architecture identified during our experimentation.

3 Results

3.1 Comparing Loss

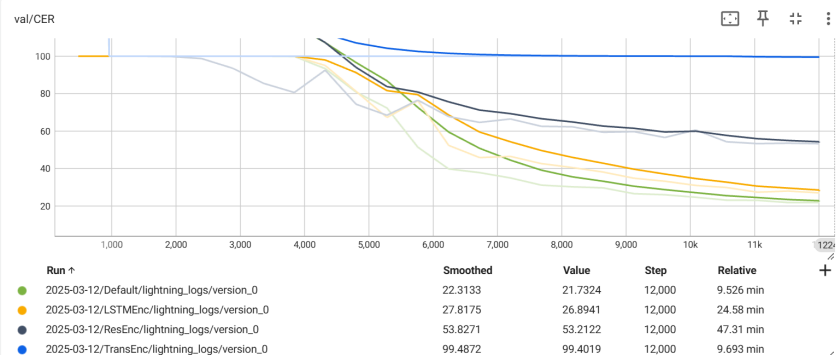

Figure 1: Encoder Val Loss Comparisons

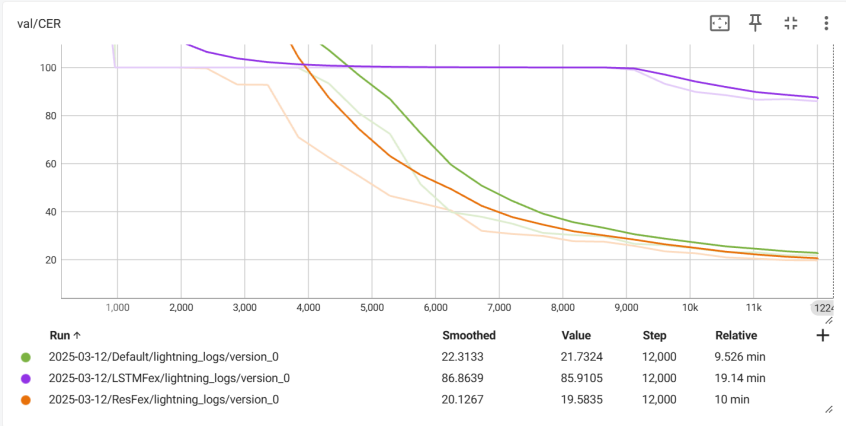

Figure 2: Feature Extractor Val Loss Comparisons

Table 1: Comparison of CER (%) on Validation and Test sets for our proposed architectures

Model	**Val CER**	**Test CER**
baseline	22.31	23.56
LSTM Encoder	26.89	28.07
ResNet Encoder	53.21	42.83
Transformer Encoder	99.48	99.81
ResFEX	**19.58**	**20.64**
LSTMFEX	86.86	84.76
ResFEX + LSTM Encoder	23.53	23.79

Note: model sizes are in the range of 3M-10M parameters

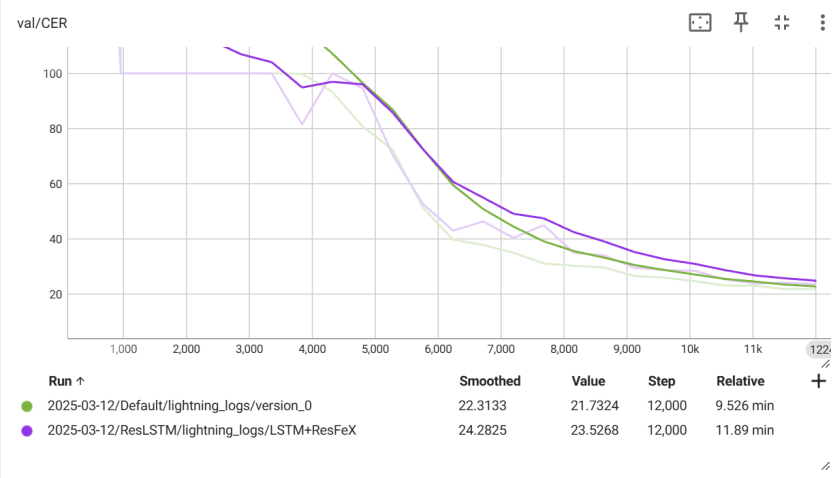

Figure 3: *baseline* vs. LSTM Encoder + ResFEX Val Comparison

4 Fine Tuning the best model

4.1 What to finetune?

With the unchanged hyperparameters, the ResFex model was achieving a Val CER of 19.58. If we refer to Figure 3, the loss began plateauing near the end and took a considerable number of epochs to learn given our constraints.

Thus we tackled the learning rate scheduler as possibly the highest yield place to focus our compute.

The problem with Cosine Annealing, it took quite a few epochs for the model to really start decreasing its CER. This made us believe that it started way too low of a learning rate. And starting the annealing earlier in training did not seem to help. In addition, simply increasing the learning rate would cause the learning rate at the beginning of the cosine function to be large.

Thus we compared the learning rate schedules in Figure 4 to test their change on our model performance.

4.2 Results

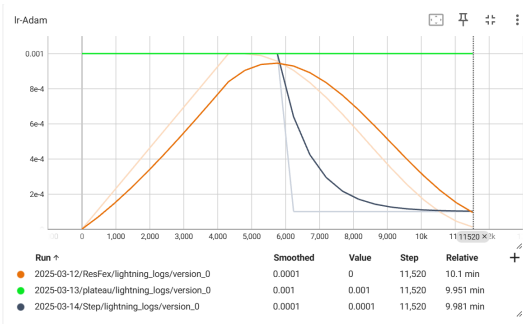

Figure 4: ResFEX LR scheduler comparison

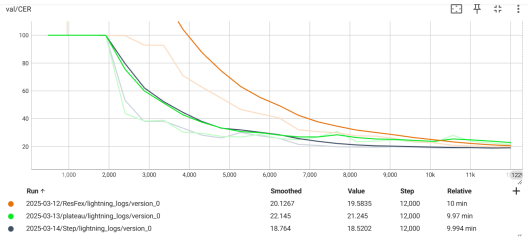

Figure 5: ResFEX LR schedule Validation Loss comparison

4.2.1 Learning Rate Graphs Interpretation

As you can see in Figure 5, the stepped and the plateaued models learned really fast, but the plateaued model achieved similar performance, as it never met the condition to lower its learning rate. The stepped model schedule lowered the learning rate earlier than the cosine model and achieve a lower CER in the end.

5 Discussion

Our experiments gave us several insights into the effectiveness of different model architectures in attempting to predict characters from typing data captured by sEMG inputs. Our results suggest that mainly the encoder and feature extractor have the most impactful effect on performance, and combining the best performing elements does not always give the best results in regards to overall model performance.

5.1 Encoder Performance Analysis

The choice of encoder architecture had the most significant impact on character error rate (CER). Our results in Table 1 and Figure 1 show a very clear and distinct ranking in terms of encoder performance. The convolutional baseline encoder preformed the best, achieving the lowest CER among all models. The LSTM Encoder followed closely, showing a very strong performance but slightly under preforming compared to the ResNet. The ResNet encoder performed quite badly. The Transformer Encoder architecture did not seem to learn the data within 25 epochs. To test that we didn't stop early, we ran the transformer encoder for 75 epochs as well and the performance was still similar ($\sim$96 CER).

We hypothesize the reason for these results is tied into the inherent temporal structure of sEMG signals. LSTMs have mechanics best suited for effectively capturing time dependent relationships between data. ResNets leverage shortcut connections to allow deep networks to propagate essential feature representations. On the other hand, LSTMs model long-term relationships using recurrent connections, making them well-suited for time-series data. We infer that the ResNet encoder preformed worse due to the lack of specific use of time features present in recurrent architectures. This is surprising, however, as we expect the residual encoder to be at least as good as the baseline CNN encoder. One potential reason for this is that the residual encoder architecture used two convolutional layers compared to a single convolutional layer within the baseline encoder, indicating a potential need for a longer training period.

In contrast, the Transformer encoder despite its strong track record in NLP and vision tasks did not generalize within the 25 epoch range for a single patient. Transformers inherently rely on self-attention to capture relationships across input data, which works well when spatial relationships such as in text or images are well-defined. However, we presume that since sEMG are capturing discrete motor neuron activations as its signal, these spatial relationships contain a weaker signal and require more data and epochs to learn. An additional detail to take into account is that transformers also traditionally require drastically more data to unlock their best performance compared to RNNs. Transformers are known to generalize well over a lot of data, and we infer our set up did not allow the self-attention mechanism to generate a dense embedding. Due to this analysis, we infer the recurrent architectures performed better within this context.

5.2 Feature Extractor Performance Analysis

In Table 1 and Figure 2, we show our feature extractor results. The LSTM Feature Extractor performed the worst, implying that the LSTM is more suited for extracting relationships between embeddings rather than the first attempt at feature extraction within the model. Another potential reason for its performance is the disturbance of the noise within the sEMG signal being modelled in recurrent architectures. Next, the baseline feature extractor performed well; we concluded that the spatial features which the CNN model can be interpreted as the shape of the signal itself. One interpretation is that this indicates distinct features within sets of motor activations which visibly look different between different typed keys and the sequence of different typed keys. The ResNet feature extractor performed best. This was somewhat surprising, given it's performance as an encoder. Our take away from this is that while the ResNet based encoder may not perform very well at learning feature representations when applied later in the pipeline, as a feature extractor it performs well. We infer this has to do with the feature gradients propagating throughout our residual block. The convolutional layers in ResNet operate on a local field, and help smooth out the data, it does not need to capture the long-term dependencies as it is only tightening the current important features of the sEMG.

5.3 Combining Encoders and Feature Extractors

Given that we were working on a limited computational budget and deadline, we reasoned that we would test our Encoders and Feature Extractors separately to find the best preforming ones. Our intuitive hypothesis was that combining the best Encoder and Feature Extractor would yield the best preforming model. However, our results in Table 1 and Figure 3 showed that the ResNet Feature extractor and the LSTM Encoder model actually preformed worse than the baseline model.

Our results show that there may exist inherent incompatibility between the way the LSTM channels handle sequential dependencies and how the ResNet processes spatial information. The LSTM are built to learn and emphasize temporal relationships and we infer that there is a trade off of spatial representation, which the ResNet and CNNs are more well suited for handling. We saw this before as well, when the LSTM was paired with the MLP, it seems it focuses too much on time dependencies and that are not well-suited for the data. The TDSConvEncoder, which was a convolutional block followed by a fully connected block, was able to take into account more of the spatial features, and then the fully connected block integrated these features into a single representation.

6 Future Work

In our future work, we hope to explore additional refinements and enhancements to the existing model architectures with which we experimented and tested to further improve their performance and generalization while maintaining a small model size.

If we had additional computational resources, we would ideally train a ResNet feature extractor combined with a Transformer encoder on a much larger dataset for an extended number of epochs. We believe this setup would better capture both short-term and long-term dependencies in sEMG signals, making it more suitable for this task. ResNets and, in particular, Transformers are known to require large amounts of data and significant computational power to achieve strong generalization. In this trade-off, Transformers typically outperform more traditional non-attention-based methods when trained on a sufficiently large dataset.

In addition, visualizing the spatial features of the sEMG signal would be an interesting task especially if you were able to relate these visualizations to the typed character and typing styles more generally. We believe this work can help gain intuition for how these networks model the data and for enhancing the networks themselves. In our work, we observed that spatial representations of temporal data seemed to work best rather than temporal or mixed models.

Another key direction for future research is fine-tuning each model variant individually to determine the optimal set of hyperparameters for each architecture. Due to computational constraints, we applied similar hyperparameters to maintain a similar size of each of our models and evaluated their performance under these fixed conditions, even though different architectures may be better suited to different hyperparameter configurations. Given the empirical nature of hyperparameter selection, we opted for only limited fine-tuning on our best-performing architectures. However, further optimizing

learning rates, dropout rates, and architectural parameters could lead to improved performance and a better understanding of the best ways to model sEMG signals.

References

[1] Sivakumar, V., Seely, J., Du, A., Bittner, S. R., Berenzweig, A., Bolarinwa, A., Gramfort, A., & Mandel, M. I. (2024).** *EMG2QWERTY: A Large Dataset with Baselines for Touch Typing using Surface Electromyography.* arXiv.[https://doi.org/10.48550/arXiv.2410.20081]

[2] Campanini, I., Disselhorst-Klug, C., Rymer, W. Z., & Merletti, R. (2020). Surface EMG in Clinical Assessment and Neurorehabilitation: Barriers Limiting Its Use. Frontiers in neurology, 11, 934. https://doi.org/10.3389/fneur.2020.00934

[3] J. Qi, G. Jiang, G. Li, Y. Sun and B. Tao, "Intelligent Human-Computer Interaction Based on Surface EMG Gesture Recognition," in IEEE Access, vol. 7, pp. 61378-61387, 2019, doi: 10.1109/ACCESS.2019.2914728. keywords: Feature extraction;Gesture recognition;Electromyography;Human computer interaction;Thumb;Time-domain analysis;Urban intelligence;human-computer interaction;sEMG;gesture recognition,

[4] Sehgal, A., & Kehtarnavaz, N. (2019). Guidelines and Benchmarks for Deployment of Deep Learning Models on Smartphones as Real-Time Apps. arXiv. https://doi.org/10.48550/arXiv.1901.02144